



- 1 **Design a Non-Deterministic Finite Automaton (NBA) for an Automatic Door System** based on sensor inputs. An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed*, *Opening*, *Open*, and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.
- 2 **Design a finite automaton to model a traffic light control system at a road intersection.** The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

StatesAlphabetTransitions

State name (e.g. q0)Add

RedSTARTLIVE

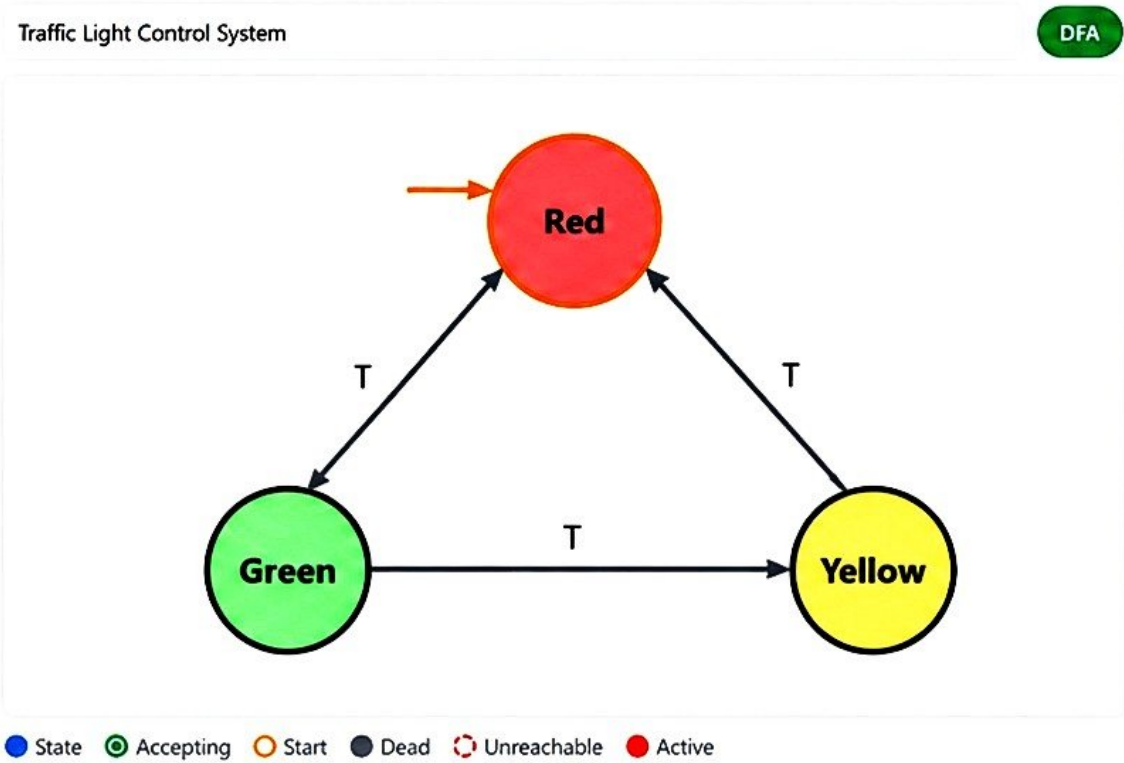
GreenLIVE

YellowLIVE

Set as startToggle accepting

RenameDelete

Tip: drag states on the canvas. Double-click one to toggle whether it is accepting.



PropertiesConvert

Deterministic (DFA)	Yes
Complete	Yes
Connected	Yes
ε-transitions	No
States / symbols	3 / 1

Unreachable states

None

Dead states

None

Missing transitions

None

Validation

The automaton is complete and valid.

INPUT STRING TTTTTT

Simulate

⏮ ⏪ ▶ Auto ⏩ ⏭

T T T T T T T

Step 6 / 6 — On 'T': (Yellow) → (Red)

ACCEPTED

Step	Input Symbol	Current State	Next State
0	—	Red	Red (start)
1	T	Red	Green
2	T	Green	Yellow
3	T	Yellow	Red
4	T	Red	Green
5	T	Green	Yellow
6	T	Yellow	Red

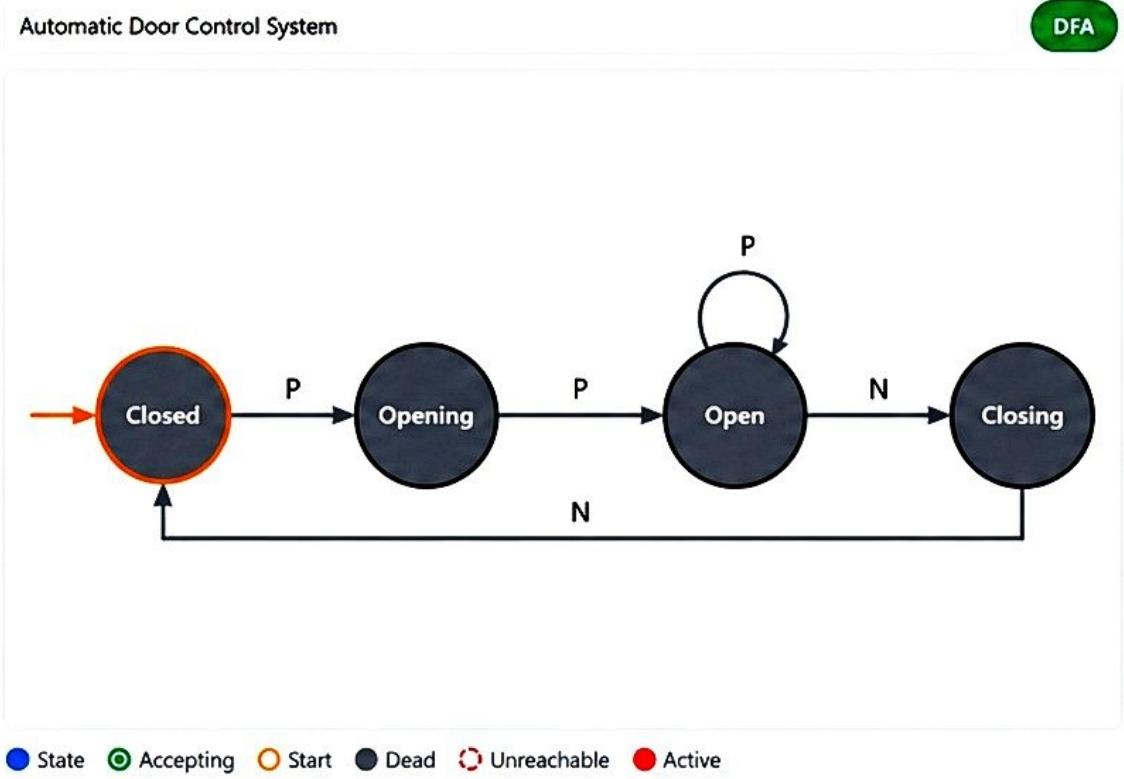
StatesAlphabetTransitions

State name (e.g. q0)Add

ClosedSTARTLIVELIVEOpeningLIVEOpenLIVEClosingLIVE

Set as startToggle acceptingRenameDelete

Tip: drag states on the canvas. Double-click one to toggle whether it is accepting.



PropertiesConvert

Deterministic (DFA)	Yes
Complete	Yes
Connected	Yes
ε-transitions	No
States / symbols	4 / 2

Unreachable statesNone

Dead statesNone

Missing transitionsNone

ValidationThe automaton is complete and valid.

INPUT STRING PPNNSimulate

Step 4 / 4 — On 'N': (Closing) → (Closed)

ACCEPTED

Step	Input Symbol	Current State	Next State
0	—	Closed	Closed (start)
1	P	Closed	Opening
2	P	Opening	Open
3	N	Open	Closing
4	N	Closing	Closed